

Урок 24. Корутины. Практика. Retrofit.

20 ноября 2020

В этом уроке рассматриваем интеграцию корутин в Retrofit

В последних версиях Retrofit была добавлена поддержка корутин. Посмотрим, как это работает.

Сначала давайте вспомним, какие у нас есть варианты использования ретрофит без корутин (и RxJava). Предполагается, что вы уже знаете, что такое ретрофит и как его использовать.

В качестве примера будем использовать сервис, возвращающий список комментариев.

```
1 interface CommentApiService {
2
3     @GET("comments")
4     fun getComments(): Call<List<Comment>>
5
6 }
```

И у нас есть ViewModel с методом fetchData, который вызывается по нажатию кнопки Refresh или какому-либо другому событию. В нем мы планируем делать вызов сервиса и получать данные:

```
1 class MainViewModel : ViewModel() {
2
3     // ...
4
5     fun fetchData() {
6         // use retrofit
7     }
8
9     // ...
10 }
```

Стандартный вариант использования ретрофит - это метод enqueue с колбэком:

```
1 fun fetchData() {
2     service.getComments().enqueue(object : Callback<List<Comment>> {
3         override fun onResponse(call: Call<List<Comment>>, response: Response<List<Comment>>) {
4             // ...
5         }
6
7         override fun onFailure(call: Call<List<Comment>>, t: Throwable) {
8             // ...
9         }
10    })
11 }
12 }
```

OnResponse нам придет результат в случае успеха, а в onFailure - ошибка в случае неудачи. Это асинхронный способ, его можно выполнять в main потоке.

Нас сейчас больше интересует другой способ, синхронный:

```
1 fun fetchData() {
2     val response = service.getComments().execute()
3
4     // ...
5 }
```

Но в main потоке так делать нельзя, потому что поток заблокируется. Надо выносить этот вызов в фоновый поток. Давайте посмотрим, как это можно сделать с помощью корутин. Так мы лучше поймем, в чем именно заключается интеграция корутин в ретрофит.

Самое простое решение - обернуть вызов в withContext. Функция withContext позволяет переключить поток выполнения кода, дождаться результата и вернуться обратно в свой поток.

```
1 val response = withContext(Dispatchers.IO) {
2     service.getComments().execute()
3 }
```

Помещаем наш синхронный код выполнения запроса в withContext и указываем, что он должен выполняться в IO потоке. withContext выполнит код и вернет нам результат в response.

Но т.к. withContext - suspend функция, мы не можем вызвать ее где угодно:

```

1 fun fetchData() {
2     val response = withContext(Dispatchers.IO) {
3         service.getComments().execute()
4     }
5     // ...
6 }
7 }

```

Этот код будет ругаться, что withContext должен быть вызван в корутине или в suspend функции.

Ок, добавим корутину:

```

1 fun fetchData() {
2     viewModelScope.launch {
3         val response = withContext(Dispatchers.IO) {
4             service.getComments().execute()
5         }
6         // ...
7     }
8 }
9 }
10 }

```

Используем viewModelScope, чтобы вызвать корутину. (Подробно о разных видах получения scope я рассказывал в Уроке 23).

Код внутри корутины выполняется в main потоке. Запрос данных внутри withContext будет выполнен в IO потоке. Main поток в это время не будет заблокирован. Как только ретрофит вернет данные, они будут помещены в response, код снова продолжит работу в main потоке и мы сможем сделать все что нужно с полученными данными.

Таким образом мы получаем данные, используя корутины вместо колбэков. Это удобнее и читабельнее, особенно если нам надо сделать несколько таких вызовов.

При этом мы можем еще улучшить читаемость кода. Всю конструкцию withContext с вызовом ретрофита давайте вынесем из корутины в отдельную suspend функцию, которую назовем getComments.

Код теперь выглядит так:

```

1 fun fetchData() {
2     viewModelScope.launch {
3         val response = getComments()
4         // ...
5     }
6 }
7 }
8 }
9 suspend fun getComments(): Response<List<Comment>> {
10     return withContext(Dispatchers.IO) {
11         service.getComments().execute()
12     }
13 }

```

Корутина стала лаконичнее и читабельнее.

Чтобы нам каждый раз не оборачивать получение данных в отдельную suspend функцию с withContext, ретрофит научился делать это сам. В этом и заключается интеграция корутин в ретрофит.

Для этого мы в интерфейсе добавляем слово suspend к методу. А Call обертку больше не используем.

```

1 interface CommentApiService {
2     @GET("comments")
3     suspend fun getComments(): List<Comment>
4 }
5 }
6 }

```

Теперь вызов метода в корутине выглядит так:

```

1 viewModelScope.launch {
2     val comments = service.getComments()
3     // ...
4 }
5 }

```

И нам не надо создавать никаких отдельных suspend методов с withContext. Ретрофит делает это под капотом. Он сам перенесет операцию в фоновый поток и вернет нам данные в поток нашей корутины.

Давайте немного расширим пример. Будем получаемые данные помещать, например, в LiveData:

```

1 val comments = MutableLiveData<List<Comment>>()
2 fun fetchData() {
3     viewModelScope.launch {
4         val commentsData = service.getComments()
5         comments.value = commentsData
6     }
7 }
8 }

```

Метод fetchData запускает корутину, которая получает данные с помощью ретрофит и помещает их в LiveData. Ну а на LiveData уже подписан какой-либо экраный элемент, который отобразит полученные данные.

Response

Чтобы поймать ошибки при получении данных, нужно просто обернуть вызов suspend функции в try-catch.

```
1 viewModelScope.launch {
2     try {
3         val commentsData = service.getComments()
4     } catch (e: Exception) {
5         // ...
6     }
7 }
```

Но как вы, возможно, знаете, ретрофит предоставляет возможность разделять обычные ошибки и HTTP ошибки (например HTTP 404). Для этого надо обернуть тип данных в Response:

```
1 interface CommentApiService {
2
3     @GET("comments")
4     suspend fun getComments(): Response<List<Comment>>
5
6 }
```

Теперь мы вместо данных будем получать объект обертку Response:

```
1 viewModelScope.launch {
2     try {
3         val response = service.getComments()
4         if (response.isSuccessful) {
5             val commentsData = response.body()
6         }
7     } catch (e: Exception) {
8         // ...
9     }
10 }
```

Если все прошло хорошо (isSuccessful), то наши данные лежат в response.body.
Если произошла HTTP ошибка, то данные о ней будут можно найти в response.
Если произошло другая ошибка, то она будет поймана в catch.